

Analyse Onglet Roles

VTTApp

Rapport d'analyse technique et recommandations

Composant RolesTab.tsx

Version de l'application: 0.709

Date d'analyse: 21 mai 2026

Fichier analyse: src/components/layout/tabs/RolesTab.tsx

Lignes de code: 354

Document confidentiel - Usage interne uniquement

Table des Matieres

1 Resume Executif	3
2 Architecture Actuelle	4
2.1 Structure du Composant	4
2.2 Gestion d'Etat	4
2.3 Flux de Donnees	5
3 Points Forts	5
4 Faiblesses Identifiees	6
4.1 Fonctionnalites Manquantes	6
4.2 Problemes UX	7
4.3 Qualite du Code	7
4.4 Risques de Securite	8
5 Comparaison avec TagsTab	8
6 Recommandations Priorisees	10
6.1 P0 - Critique	10
6.2 P1 - Important	11
6.3 P2 - Amelioration	11
6.4 P3 - Futur	12
7 Estimation d'Effort	12
8 Conclusion	13

1. Resume Executif

Ce document presente une analyse approfondie du composant RolesTab.tsx de l'application VTTApp. L'objectif est d'identifier les forces, faiblesses, et opportunités d'amélioration de l'onglet de gestion des rôles, en le comparant avec l'onglet TagsTab qui sert de référence en termes de fonctionnalités implémentées.

— Etat Actuel en Bref

- 354 lignes de code - composant de complexité moyenne
- 2 sections principales: Creation et Liste des rôles
- Groupement des rôles par équipe avec expand/collapse
- Drag-and-drop pour le reordonnement des sections
- Checkbox de sélection pour distribution aléatoire
- Actions par rôle: dupliquer, modifier, supprimer

— Problemes Majeurs Identifies

- Aucune confirmation avant suppression (risque de perte de données)
- Pas de recherche/filtrage des rôles
- Pas de tri (alphabétique, par équipe, par date)
- Pas d'import/export JSON
- Pas de modèles prédéfinis de rôles
- Pas de statistiques d'utilisation des rôles
- Pas de comptage de rôles par équipe dans l'en-tête
- Pas d'opérations en masse (bulk)
- Description et image non affichées dans la liste
- Pas de persistance de l'ordre des sections (localStorage)

— Score Global

5.5 / 10

Le composant est fonctionnel mais manque de nombreuses fonctionnalités présentes dans TagsTab

2. Architecture Actuelle

— 2.1 Structure du Composant

Le composant RolesTab.tsx (354 lignes) est structure comme suit:

Arborescence du composant

```
RolesTab.tsx (354 lignes)
|-- DynamicColor (l.10-22): Wrapper pour couleurs dynamiques
|-- SortableSection (l.24-70): Section drag-and-drop
|-- RolesTab (l.72-354): Composant principal
    |-- State: newRoleName, newRoleColor, expandedTeams
    |-- State: sectionOrder, openSections
    |-- useMemo: rolesByTeam (groupement par equipe)
    |-- renderCreateRole(): Formulaire de creation
    |-- renderRolesList(): Liste groupée par equipe
```

— 2.2 Gestion d'Etat

Le composant utilise un melange d'etat local (useState) et de store global (Zustand):

Extrait de gestion d'etat

```
// Etat local (RolesTab.tsx:74-86)
const [newRoleName, setNewRoleName] = useState('');
const [newRoleColor, setNewRoleColor] = useState('#3b82f6');
const [expandedTeams, setExpandedTeams] = useState<Record<string, boolean>>({});
const [sectionOrder, setSectionOrder] = useState(['createRole', 'rolesList']);
const [openSections, setOpenSections] = useState({ createRole: true, rolesList: true });

// Store global (RolesTab.tsx:73)
const { roles, teams, setEditingEntity, addRole, updateRole, deleteRole } = useVttStore();
```

Remarque: L'ordre des sections n'est PAS persiste en localStorage, contrairement a TagsTab (TagsTab.tsx:118-120, 128).

— 2.3 Flux de Donnees

Interface Role complete (types/index.ts:61-81):

Interface Role (types/index.ts:61-81)

```
interface Role {
  id: EntityId;
  name: string;
  color: string;
  lives: number;
  isUnique: boolean;
  teamId: EntityId | null;
  tags: TagModel[];
  imageUrl?: string;
  seenAsRoleId?: EntityId | null;
  seenInTeamId?: EntityId | null;
  description?: string;
  isSelectableForDistribution?: boolean;
  distributionQuantity?: number;
  smartphoneImageStyle?: 'circle' | 'square' | 'original' | 'background' | 'none';
  defaultCount?: number;
  minCount?: number;
  maxCount?: number;
  isFiller?: boolean;
  isMinMandatory?: boolean;
}
```

Actions du store (store/index.ts:667-675):

Actions store (store/index.ts:667-675)

```
addRole: (roleData) => set((state) => ({
  roles: [...state.roles, { ...roleData, id: uuidv4() }]
})),
updateRole: (id, updates) => set((state) => ({
  roles: state.roles.map(r => r.id === id ? { ...r, ...updates } : r)
})),
deleteRole: (id) => set((state) => ({
  roles: state.roles.filter(r => r.id !== id)
})))
```

3. Points Forts

— Architecture et Design

- Separation claire entre creation et liste des roles (SRP respecte)
- Utilisation de useMemo pour le groupement par equipe (performance)
- Composant DynamicColor pour gestion de couleurs dynamiques (RolesTab.tsx:10-22)
- Sections reordonnables via drag-and-drop (@dnd-kit)

— Fonctionnalités Implémentées

- Creation de role avec nom et couleur
- Groupement automatique par equipe avec 'Sans Equipe' par default
- Expand/collapse par equipe avec bouton 'Tout deployer/replier'
- Checkbox isSelectableForDistribution pour chaque role
- Aperçu de couleur du role dans la liste
- Affichage des PV (lives) et unicite (Unique/Multiple)
- Compteur de roles selectionnes dans l'en-tete (RolesTab.tsx:308)
- Duplication de role avec suffixe '(Copie)'
- Edition via EditingModal (setEditingEntity)
- Suppression directe (mais sans confirmation)

— Integration avec le Reste de l'Application

- Les roles sont utilises dans 139+ references dans le codebase
- Integration avec Canvas pour l'affichage des couleurs de role
- Integration avec RightPanel pour la distribution aleatoire
- Integration avec EditingModal pour l'edition complete
- Integration avec le moteur d'actions (role-team-effects.ts)
- Selectors optimises dans store/selectors.ts (selectRoles, selectRoleById, selectRolesByTeam)

4. Faiblesses Identifiees

— 4.1 Fonctionnalites Manquantes (vs TagsTab)

Le tableau suivant compare les fonctionnalites entre RolesTab et TagsTab:

Fonctionnalite	RolesTab	TagsTab	Ecart
Recherche/Filtre	NON	OUI	Majeur
Tri (A-Z/Date)	NON	OUI	Majeur
Mode de vue	NON	OUI (2 modes)	Majeur
Import JSON	NON	OUI	Majeur
Export JSON	NON	OUI	Majeur
Modeles predefinis	NON	OUI	Majeur
Dashboard stats	NON	OUI	Majeur
Confirmation delete	NON	OUI	Critique
Compteur usage	NON	OUI	Important
Drag & drop items	NON	OUI	Important
Persistence ordre	NON	OUI	Important
Tri dans groupes	NON	OUI	Mineur
Validation doublon	NON	OUI	Important
Creation inline equipe	NON	OUI	Mineur

— 4.2 Problemes UX

- **Suppression sans confirmation: Un clic accidentel supprime un role definitivement**

Reference: RolesTab.tsx:285-291 - deleteRole(role.id) appele directement sans modal de confirmation.

- **Pas de feedback visuel lors de la creation d'un role en doublon**

Contrairement a TagsTab qui affiche un message d'erreur (TagsTab.tsx:168-169), RolesTab cree silencieusement un doublon.

- **La liste des roles n'affiche pas la description du role**

L'interface Role possede un champ description (types/index.ts:72) mais il n'est pas affiche dans la liste. L'utilisateur doit ouvrir EditingModal pour la voir.

- **Pas d'aperu de l'image du role dans la liste**

Le champ imageUrl existe (types/index.ts:69) mais n'est pas utilise dans RolesTab. TagsTab affiche l'icone du tag dans la liste.

- **Les tags attaches au role ne sont pas visibles dans la liste**

Le champ tags: TagModel[] (types/index.ts:68) existe mais n'est pas affiche. L'utilisateur ne sait pas quels tags sont associes sans ouvrir l'edition.

- **Pas de comptage de roles par equipe dans l'en-tete de groupe**

L'en-tete d'equipe affiche seulement (selectifs/total) mais pas le nombre total de roles de l'equipe de maniere explicite.

- **Pas de creation d'equipe inline depuis l'onglet Roles**

Si aucune equipe n'existe, l'utilisateur doit aller dans l'onglet Equipes pour en creer une.

— 4.3 Qualite du Code

- **Composant monolithique de 354 lignes**

Le composant RolesTab combine creation, liste, drag-and-drop, et gestion d'etat dans un seul fichier. TagsTab (530 lignes) est plus long mais mieux structure avec des sous-composants memoises (TagListItem avec React.memo).

- **Pas de React.memo sur les elements de liste**

Chaque role est rendu dans un .map() sans memoisation. TagsTab utilise React.memo sur TagListItem (TagsTab.tsx:51).

- **Pas de validation de nom en doublon a la creation**

handleAddRole (RolesTab.tsx:127-146) ne verifie pas si un role du meme nom existe deja.

- **Pas de gestion d'erreur sur les actions du store**

Les appels addRole, updateRole, deleteRole n'ont pas de gestion d'erreur ou de try/catch.

- **Pas de lazy loading ou virtualisation pour les longues listes**

Bien que l'application utilise @tanstack/react-virtual (package.json:19), RolesTab ne l'utilise pas. Pour 50+ roles, les performances pourraient se degrader.

— 4.4 Risques de Securite et Integrite des Donnees

- **Suppression en cascade non geree**

deleteRole (store/index.ts:673-675) supprime uniquement le role. Si des joueurs ont ce role (Player.roleId), leurs references deviennent invalides. TagsTab gere le nettoyage des tags lors de la suppression.

- **Pas de traçage de l'utilisation des roles**

Contrairement a selectTagUsageCount (selectors.ts:156-161), il n'existe pas de selectRoleUsageCount. Impossible de savoir si un role est assigne a des joueurs avant suppression.

5. Comparaison Detaillee avec TagsTab

— 5.1 Analyse des Ecart de Fonctionnalites

TagsTab sert de reference dans cette application. Voici une analyse detaillee de chaque ecart:

— Recherche et Filtrage

TagsTab implemente une barre de recherche complete (TagsTab.tsx:113, 153-162, 374-379):

Implementation recherche TagsTab

```
// TagsTab.tsx:153-162
const filteredTagsByCategory = useMemo(() => {
  if (!searchQuery.trim()) return sortedTagsByCategory;
  const query = searchQuery.toLowerCase();
  const filtered: Record<string, typeof tags> = {};
  Object.entries(sortedTagsByCategory).forEach(([catId, catTags]) => {
    const matching = catTags.filter(t => t.name.toLowerCase().includes(query));
    if (matching.length > 0 || catId === 'no-category') filtered[catId] = matching;
  });
  return filtered;
}, [sortedTagsByCategory, searchQuery]);
```

RolesTab: Aucune fonctionnalite de recherche. Pour 20+ roles, l'utilisateur doit parcourir manuellement chaque equipe.

— Tri et Organisation

TagsTab offre le tri alphabetique et par date (TagsTab.tsx:114, 145-151, 298-300):

Implementation tri TagsTab

```
// TagsTab.tsx:145-151
const sortedTagsByCategory = useMemo(() => {
  const sorted: Record<string, typeof tags> = {};
  Object.entries(tagsByCategory).forEach(([catId, catTags]) => {
    sorted[catId] = [...catTags].sort((a, b) =>
      sortBy === 'name' ? a.name.localeCompare(b.name, 'fr') : 0
    );
  });
  return sorted;
}, [tagsByCategory, sortBy]);
```

RolesTab: Les roles dans chaque equipe ne sont tries selon aucun critere. L'ordre depend de l'ordre d'insertion dans le store.

— Import/Export

TagsTab implemente l'export JSON (TagsTab.tsx:195-201) et l'import JSON (TagsTab.tsx:203-214):

Implementation export TagsTab

```
// TagsTab.tsx:195-201
const handleExportTags = useCallback(() => {
  const data = { version: '1.0', tags, tagCategories, exportedAt: new Date().toISOString() };
  const blob = new Blob([JSON.stringify(data, null, 2)], { type: 'application/json' });
  const url = URL.createObjectURL(blob);
  const a = document.createElement('a');
  a.href = url; a.download = 'tags-export.json';
  a.click(); URL.revokeObjectURL(url);
}, [tags, tagCategories]);
```

RolesTab: Aucune fonctionnalité d'import/export. Impossible de sauvegarder/restaurer une configuration de rôles.

— Modeles Predefinis

TagsTab utilise PREDEFINED_TAG_TEMPLATES (TagsTab.tsx:9, 222-236) permettant d'importer des tags standards en un clic.

RolesTab: Aucun modèle prédefini. Chaque rôle doit être créé manuellement.

— Dashboard Statistiques

TagsTab offre un dashboard complet (TagsTab.tsx:441-527) avec:

- **Nombre total de tags, categories, tags dans distributeur**
- **Top 5 des tags les plus utilisés avec barres de progression**
- **Repartition par categorie**
- **Tags avec auto-suppression**

RolesTab: Aucune statistique. Impossible de voir la répartition des rôles par équipe, le nombre de rôles uniques vs multiples, etc.

— Confirmation de Suppression

TagsTab implémente un modal de confirmation (TagsTab.tsx:123, 188-193, 389-402):

Implementation confirmation TagsTab

```
// TagsTab.tsx:123
const [deleteConfirm, setDeleteConfirm] = useState<
  { type: 'tag' | 'category'; id: string; name: string } | null
>(null);

// TagsTab.tsx:188-193
const handleRequestDelete = useCallback(
  (type, id, name) => setDeleteConfirm({ type, id, name }), []
);
```

RolesTab: Suppression directe sans confirmation (RolesTab.tsx:285-291). Risque élevé de suppression accidentelle.

— Persistance de l'Ordre des Sections

TagsTab persiste l'ordre des sections en localStorage (TagsTab.tsx:118-120, 128):

Implementation persistance TagsTab

```
// TagsTab.tsx:118-120
const [sectionOrder, setSectionOrder] = useState(() => {
  try {
    const saved = localStorage.getItem('tagsTabSectionOrder');
    return saved ? JSON.parse(saved) : ['createCategory', 'createTag', 'tagList'];
  } catch { return ['createCategory', 'createTag', 'tagList']; }
});

// TagsTab.tsx:128
useEffect(() => {
  try { localStorage.setItem('tagsTabSectionOrder', JSON.stringify(sectionOrder)); }
  catch {}
}, [sectionOrder]);
```

RolesTab: L'ordre des sections est perdu à chaque rechargement de page.

6. Recommandations Priorisees

— 6.1 P0 - Critique (A implementer immediatement)



P0 - Critique

- **Modal de confirmation de suppression**

Ajouter un modal identique a TagsTab avant chaque suppression de role. Impact: evite la perte accidentelle de donnees. Reference: s'inspirer de TagsTab.tsx:389-402.

- **Verification de doublon a la creation**

Ajouter une validation du nom avant creation. Si un role du meme nom existe, afficher un message d'erreur. Reference: s'inspirer de TagsTab.tsx:168-169.

- **Gestion de la suppression en cascade**

Avant de supprimer un role, verifier si des joueurs y sont assignes. Si oui, afficher un avertissement et proposer de reassigner les joueurs a 'null'. Reference: creer un selectRoleUsageCount similaire a selectTagUsageCount (selectors.ts:156-161).

— 6.2 P1 - Important (A implementer rapidement)



P1 - Important

- **Barre de recherche/filtrage**

Ajouter une barre de recherche identique a TagsTab. Filtrer les roles par nom. Reference: TagsTab.tsx:374-379.

- **Tri alphabetique des roles dans chaque equipe**

Ajouter un bouton de tri A-Z/Date. Trier les roles par nom par default. Reference: TagsTab.tsx:145-151.

- **Import/Export JSON des roles**

Permettre l'export de tous les roles en JSON et l'import depuis un fichier. Reference: TagsTab.tsx:195-214.

- **Persistence de l'ordre des sections en localStorage**

Sauvegarder et restaurer l'ordre des sections. Reference: TagsTab.tsx:118-120, 128.

- **Compteur de roles par equipe dans l'en-tete**

Afficher le nombre total de roles dans chaque en-tete d'equipe, pas seulement le ratio selectifs/total.

- **Affichage de la description dans la liste**

Afficher la description du role sous le nom, en texte grise italique, si elle existe. Reference: TagsTab.tsx:78.

— 6.3 P2 - Amelioration (A planifier)



P2 - Amelioration

- **Dashboard de statistiques des roles**

Creer un modal dashboard avec: total roles, repartition par equipe, roles uniques vs multiples, roles dans distribution, top roles les plus assignes. Reference: TagsTab.tsx:441-527.

- **Modeles predefinis de roles**

Creer un fichier role-templates.ts avec des roles standards (Loup, Voyante, Guerisseur, etc.) importables en un clic.

- **Mode de vue compact/detaille**

Permettre de basculer entre une vue detaillee (actuelle) et une vue compacte (nom + couleur uniquement). Reference: TagsTab.tsx:115, 51-96.

- **Memoisation des elements de liste avec React.memo**

Extraire le rendu de chaque role dans un composant RoleListItem memoise. Reference: TagsTab.tsx:51.

- **Aperçu de l'image du role dans la liste**

Afficher une miniature de l'image du role si imageUrl est defini.

- **Affichage des tags dans la liste**

Afficher les badges des tags associes au role dans la liste.

— 6.4 P3 - Futur (Nice to have)



P3

- Futur

- **Drag-and-drop des roles entre equipes**

Permettre de deplacer un role d'une equipe a une autre par glisser-deposer. Reference: TagsTab.tsx:216-220, 319.

- **Operations en masse (bulk)**

Selection multiple de roles pour appliquer des actions en masse: changer d'equipe, activer/desactiver distribution, supprimer.

- **Creation d'equipe inline depuis l'onglet Roles**

Ajouter un bouton '+ Equipe' dans la liste pour creer une equipe sans changer d'onglet.

- **Virtualisation pour les longues listes**

Utiliser @tanstack/react-virtual pour optimiser le rendu de 50+ roles.

- **Undo/Redo pour les actions sur les roles**

L'application utilise deja zundo (package.json:31) pour l'undo global. Verifier que les actions sur les roles sont bien trackees.

7. Estimation d'Effort

— 7.1 Estimation par Recommandation

Priorite	Recommandation	Complexite	Estimation
P0	Modal confirmation suppression	Faible	1-2h
P0	Verification doublon creation	Faible	1h
P0	Suppression en cascade	Moyenne	3-4h
P1	Barre de recherche	Faible	2h
P1	Tri alphabetique	Faible	1-2h
P1	Import/Export JSON	Moyenne	3-4h
P1	Persistence localStorage	Faible	1h
P1	Compteur roles par equipe	Faible	1h
P1	Affichage description	Faible	1h
P2	Dashboard statistiques	Moyenne	4-6h
P2	Modeles predefinis	Moyenne	3-4h
P2	Mode vue compacte	Moyenne	2-3h
P2	React.memo RoleListItem	Faible	1-2h
P2	Apercu image role	Faible	1-2h
P2	Affichage tags liste	Faible	1-2h
P3	Drag-drop entre equipes	Elevee	6-8h
P3	Operations en masse	Elevee	6-8h
P3	Creation equipe inline	Moyenne	3-4h
P3	Virtualisation liste	Moyenne	3-4h
P3	Undo/Redo verification	Faible	1h

— 7.2 Estimation Totale par Priorite

Priorite	Nb Items	Temps Minimum	Temps Maximum
P0 - Critique	3	5h	7h
P1 - Important	6	7h	11h
P2 - Amelioration	6	12h	19h
P3 - Futur	5	19h	25h
TOTAL	20	43h	62h

— 7.3 Plan de Mise en OEuvre Recommande

Phase 1 (Semaine 1) - P0: Stabilisation et securite des donnees

- **Modal de confirmation de suppression**
- **Verification de doublon a la creation**

- **Gestion de la suppression en cascade**

Objectif: Eliminer les risques de perte de donnees.

Phase 2 (Semaine 2) - P1: Productivite utilisateur

- **Barre de recherche et tri**
- **Import/Export JSON**
- **Persistence localStorage et compteurs**
- **Affichage de la description**

Objectif: Rendre l'onglet Roles aussi fonctionnel que TagsTab.

Phase 3 (Semaine 3-4) - P2: Experience utilisateur

- **Dashboard statistiques**
- **Modeles predefinis**
- **Mode de vue et memoisation**

Objectif: Offrir une experience premium.

Phase 4 (Semaine 5+) - P3: Fonctionnalites avancees

- **Drag-drop entre equipes**
- **Operations en masse**

Objectif: Fonctionnalites power-user.

8. Conclusion

— Synthèse

Le composant RolesTab.tsx est fonctionnel et integre correctement avec le reste de l'application VTApp. Cependant, il presente un decalage significatif par rapport a TagsTab en termes de fonctionnalites et d'experience utilisateur.

Les 20 recommandations identifiees couvrent un spectre allant de la securite des donnees (P0) aux fonctionnalites avancees (P3), avec une estimation totale de 43 a 62 heures de developpement.

— Priorites Recommandees

1. Implementer les 3 elements P0 dans la semaine pour securiser les operations de suppression.
2. Aligner les fonctionnalites de base avec TagsTab (P1) pour une experience utilisateur coherente.
3. Ajouter les fonctionnalites differentiantes (P2) pour offrir une valeur ajoutee.
4. Planifier les fonctionnalites avancees (P3) selon les retours utilisateurs.

— References Techniques

Fichiers de reference

Fichiers analyses:

- src/components/layout/tabs/RolesTab.tsx (354 lignes)
- src/components/layout/tabs/TagsTab.tsx (530 lignes)
- src/types/index.ts (Role: lignes 61-81)
- src/store/index.ts (Actions: lignes 667-675)
- src/store/selectors.ts (Selectors: lignes 24, 113-114, 137-138)
- src/components/EditingModal.tsx (Edition role: lignes 503-864)
- src/components/layout/RightPanel.tsx (Distribution: lignes 119-441)
- src/components/layout/Canvas.tsx (Affichage: lignes 1315-1587)
- src/lib/action-engine/effects/role-team-effects.ts
- src/hooks/useCallOrder.ts (lignes 44-45)

— Metriques du Composant

Metrique	Valeur	Benchmark TagsTab
Lignes de code	354	530
Sous-composants	2	3 (SortableSection, TagListItem)
Hooks useState	5	13
Hooks useMemo	2	5
Hooks useCallback	0	7
Hooks useEffect	0	1
React.memo	0	1 (TagListItem)
Fonctionnalites	~10	~20
Persistence	Aucune	localStorage